# Multi-Robot SLAM Mapping with Multi-Arm Bandits

Gianna Bowden, Student, Arizona State University

***Abstract*—The use of multiple robots in a 2D SLAM mapping application is investigated within this report. The multi-robot system is composed of only two robots and is able to collaboratively update a shared lidar map of environment data. The multi-arm bandit algorithm was implemented with four strategy options. These strategies, or arms, consist of furthest, nearest, spread, or random options. This is used to determine which frontier a robot travels to and maps.**

**The mapping strategy was successfully implemented and the bandit favored the furthest option; this had the best reward even with a lower success rate.**

***Index Terms*—SLAM, Mapping, Bandit, Multi-Robot**

## I. Introduction

MULTI Robot systems are used in many different applications. Some of these applications include Search and Rescue, Agriculture, Construction, Planetary Exploration, and Warehouse logistics. Using multiple robots allows for a potential decrease in the time it takes to complete a task, an increase in the overall range of performance, and improved task resilience.

These systems either fall into the category of centralized or decentralized control. There is either a central entity that provides control for the robots, or they are decentralized, where each robot is responsible for control and there is no central entity. Often, multi-robot systems and control applications take inspiration from nature and biological patterns. Consider, for instance, when a school of fish move in sync, a flock of birds, or how termites collectively create massive structures.

Specifically, this project looks at the use of a multi-robot system applied to mapping of a warehouse environment. The robots use 2D SLAM and map the environment, publishing the maps to a shared map of the combined environment. The goal is to map the environment together and attempt to optimize some of the exploration behavior to map the region more quickly.

## II. Models

In order to point the robots in the direction of the frontier they should move towards, gradient descent control was implemented. This framework utilizes both repulsive and attractive fields [4], the robots' velocity is descending the potential field. The goals the robots travel to are valleys in the field, and obstacles are treated as peaks. At a high level, the equation is as follows.

$$U_{art}(x) = U_{x_d}(x) + U_O(x) \tag{1}$$

Term $x_d$ represents the goal and $O$ is the obstacle [4]. Taking the gradient of each of these terms provides the attractive force to reach $x_d$ and the force that induces repulsion. For this report, the gradient is not directly computed. This is simplified by just computing the force vectors. There are issues with local minima, and a robot could get stuck where the forces balance out.

The multi-arm bandit is modeled off of the UCB1 policy in [2]. The equation for this model is as follows.

$$\bar{x}_j + \sqrt{\frac{2\ln n}{n_j}} \tag{2}$$

where $\bar{x}_j$ is the average reward obtained from the machine $j$, $n_j$ is the number of times the machine is played, and $n$ is the overall play count [2]. Henceforth, the UCB1 algorithm will be referred to as the bandit.

For frontier mapping, the idea is any cell that is open and adjacent to an unknown one is a frontier edge [5].

### A. Frontier Detection

Frontier detection consists of finding the unexplored regions around a robot based on the combined map. This is the boundary between what is explored and what is unexplored. A cell is a frontier when it is free but has a neighbor cell that is still unknown. These are frontier regions; any group that does not meet the minimum number of unexplored cells is discarded. Each surviving region's centroid is a candidate goal point.

The detection checks the shared map at each iteration and re-runs to detect any frontiers. This map is updated as the robots explore so it is important that this consistently updates.

These points are provided to the explorer. This filters out the potential frontiers and determines what is actually feasible for the robots. These checks include determining if the point is too close to the robot, claimed by another robot, or to ensure a point is not within an obstacle.

Both the Frontier Detection and the explorer scripts rely on the merged map data to find frontiers and the known state of cells.

### B. Controller Design

The explorer produces a goal point a robot should drive to, robot by robot so robots do not get assigned the same frontier. There are four strategies employed to determine the ideal frontier. The following strategies include: nearest, farthest, random, and spread.

- Nearest: Closest Frontier to the robot
- Furthest: Farthest Frontier from the robot
- Random: Any Frontier, random from the available list
- Spread: Frontier Farthest from Other Robot, encouraging separation

The controller design utilizes a gradient descent field. The assigned frontier acts as an attractive force. In order to compute this, the controller needs to understand the current location of the robot, what is around the robot, and the goal frontier it is traveling towards. The outputs of the controller are velocity and the turning rate.

Once the robot reaches the desired frontier point, a flag is set to the explorer and the explorer sets a new goal. Since there is potential for a robot to get stuck at a minimum, a timeout is implemented. If a robot does not reach a goal within a desired time, the goal is abandoned and it is reassigned a new frontier.

### C. SLAM Map Merging

For this project, 2D SLAM mapping was utilized. The code used for the SLAM portion of this assignment is leveraged from a ROS2 package called `slam_toolbox` [1] posted on a public repository. Generally, as robots move we know their position and lidar data is accumulated. This combined with position data allows us to determine a rough map of an area. Scans are matched against previous ones and update the map over time.

Each robot runs its own `slam_toolbox` and does not share scans or align to other robots. To bridge this gap, the robots publish their maps to a shared combined map via the map merger script where the spawn location of each robot is known. This combined with the robots' maps produces an overarching, combined 2D map of an area over time.

### D. Multi-Arm Bandit

This is where all of the simulation comes together. The arms of the bandit consist of the previously mentioned strategies; nearest, furthest, random, and spread. The UCB1 formula [2], accounts for the number of pulls for a specific strategy arm, pulls for all arms, and the reward associated with it. This is a piece of the explorer, when UCB1 is not employed, this portion of the script is skipped and one of the strategies is continuously employed.

When the UCB1 runs, each strategy is scored and the one that has the highest score is used for the frontier. One of the nuances of this algorithm is that at start up, each of the arms needs to be run once to be initialized with a value.

## III. Analysis and Simulation

The simulation for this system was completed within Gazebo and RViz. The simulation generally only focused on two robots, completing the multi-robot system mapping effort.

### A. Simulation Scenarios

To assess the performance of the simulation and controller design, two scenarios were tested.

The first scenario was of a smaller warehouse [6]. This warehouse has shelving, but it is lifted off the ground and lets the robots traverse under them easily.


Fig. 1: Small Warehouse

The second scenario was a larger warehouse [6]. Within this warehouse, there are pallets on the ground. There are beams that intersect the ground in addition to many aisles that sometimes are only one way accessible. This was leveraged because this allowed us to see the mapping efficiency of the two-robot system clearly in a more complicated and larger environment.


Fig. 2: Large Warehouse

Since the Multi-arm bandit pulls arms from either random, nearest, furthest, or spread, each of these pulls were tested independently in addition to the multi-arm bandit.

### B. Simulation Analysis

A simulation was run for each arm strategy, in addition to the multi-arm bandit to see how they all compare.

For the first scenario, each simulation was run for approximately 5 minutes. The robots explored the area and mapped it. The figure below shows the coverage of each strategy arm over time.

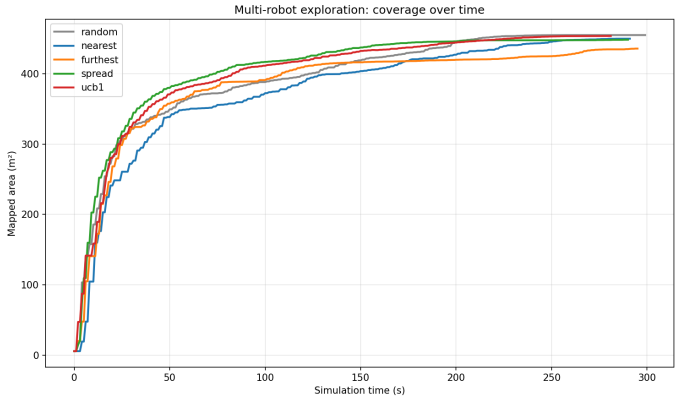


Fig. 3: Coverage over Time

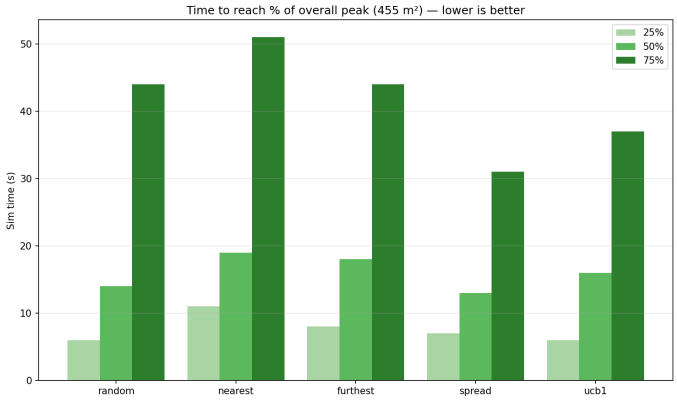


Fig. 5: Percentage of Map Discovered

Below, the information related to the arms pulled for the bandit is provided.

| Arm | Pulls | Mean Reward | Success Rate |
|---|---|---|---|
| Random | 4 | 28084 | 0.75 |
| Nearest | 3 | 25666 | 0.67 |
| Furthest | 4 | 25570 | 0.75 |
| Spread | 2 | 6191 | 0.5 |

The reward is the amount of new cells determined in exploring the frontier, the mean reward is the reward divided by the number of pulls. Success is if the robot arrived, and did not abandon the goal, divided by the number of pulls.

In the table, spread had the worst performance in the bandit with only 50% success. The other pulls did not have a lot of variability, and we can see similar rewards and success for the furthest and random. Additionally, the SLAM Map generated is seen below.

Fig. 4: Small Warehouse SLAM Map

In this simulation, all the arms perform similarly but the best is achieved by spread, followed by the bandit. It is clear that the lidar immediately is able to map a lot of the area for all the runs. The spread run pulled the arm about 17 times and a mean reward of 17k cells at a rate of about 1.5 m/s$^2$. The nearest appears to be the worst, with about 40 pulls and a mean of 7k cells.

Figure 5 shows the time to map about 25%, 50%, and 75% of the area. Spread mapped 75% of the area in about 30 seconds.

Even though the UCB had a slightly better mapping efficiency, the spread mapped more of the area in the shortest time.

In the second scenario, a larger warehouse was used to further test the capabilities of the robots. Figure 6 shows that even though spread maps the fastest within 150 seconds, the bandit generally performs the best over time.

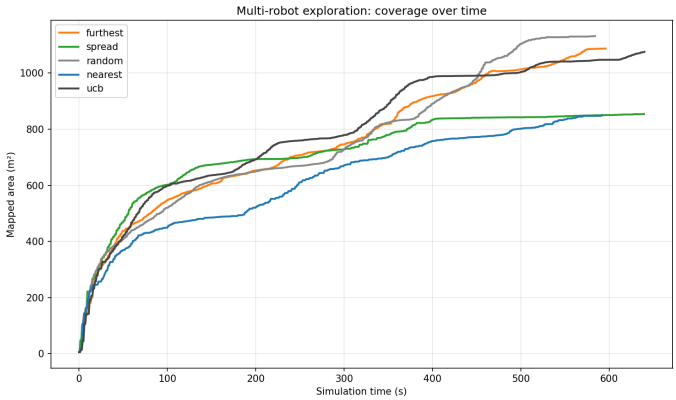


Fig. 6: Percentage of Map Discovered

Additionally, the figure below shows the mapping efficiency. Interestingly enough, random had a slightly better rate than furthest.

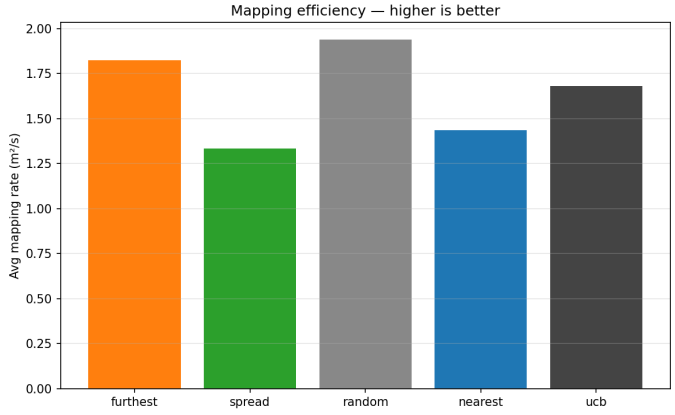


Fig. 7: Percentage of Map Discovered

In this scenario, the bandit explored each arm about twice. The nearest strategy was the most greedy with about 250 pulls but only mean reward of 2.7k, but had a high success rate. The table below shows the pulls for each arm of the bandit.

| Arm | Pulls | Mean Reward | Success Rate |
|---|---|---|---|
| Random | 6 | 37, 990 | 0.83 |
| Nearest | 2 | 14,503 | 1.0 |
| Furthest | 12 | 41,453 | 0.75 |
| Spread | 2 | 14,397 | 0.5 |

The bandit focused the pulls on the furthest strategy as it had the highest mean reward and had a lower success rate. Nearest had a 100% success rate but has a lower reward. In general with the bandit, there seems to be a trade-off between risk and reward in the methodology that does provide higher performing trends.

The final map compiled in SLAM is as follows.

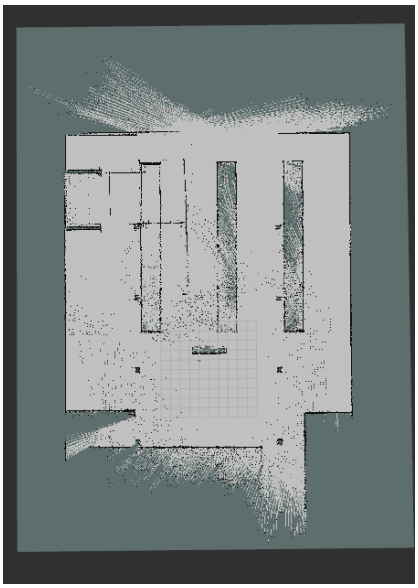

Fig. 8: Large Warehouse SLAM Map

## IV. Conclusion

Overall, the multi robot system was successfully implemented and was able to use SLAM to map the shared environment. The multi-arm bandit was also implemented and there are noticeable differences between the various implemented strategies.

Challenges encountered include occasional issues with the robots hitting points where they cannot move. Often, after some time spent mapping the large warehouse, robots would easily get stuck in tight corners and would become unable to move. Additionally, consideration of inter-robot collision avoidance had to be made since robots would get stuck running into each other.

Please note that Claude AI [3] resources were used in developing the algorithms in this code base, any errors encountered during development, and plot generation. Additionally, Claude AI, Opus 4.7, was used to aid in compiling all relevant information and resources for ROS2, Nav2, SLAM, and UCB for implementation.

## V. Further Work

Future development upon this work would include further assessment of performance over many more runs of the different strategies. Additionally, the increase in the number of robots should be investigated in relation to the strategies.

## References


[1] S. Macenski, https://github.com/SteveMacenski/slam_toolbox/tree/ros2

[2] Auer, P., Cesa-Bianchi, N. and Fischer, P. Finite-time Analysis of the Multiarmed Bandit Problem. Machine Learning 47, 235–256 (2002). https://doi.org/10.1023/A:1013689704352

[3] Anthropic. (2025). Claude (Claude Opus 4.7) [Large language model]. https://claude.ai

[4] Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. International Journal of Robotics Research, 5(1), 90–98

[5] Yamauchi, B. (1997). A frontier-based approach for autonomous exploration. In Proceedings of IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA), pp. 146–151.

[6] https://github.com/ros-navigation/nav2_minimal_turtlebot_simulation

[7] https://github.com/gciaglia/multirobotSLAM-ses598-finalproject

[8] `nav2_bringup`: https://github.com/ros-navigation/navigation2/blob/humble/nav2_bringup/README.md

[9] Package Creation Instructions: https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html